

# **Object Oriented Programming**



**SPRING 2026**

*Lecture 3 of 16*

*Engr. Majid Kaleem*

# WEEKLY AGENDA

| TENTATIVE WEEKLY DATES                                                                 |                                             | TENTATIVE TOPICS                                                               |
|----------------------------------------------------------------------------------------|---------------------------------------------|--------------------------------------------------------------------------------|
| 1                                                                                      | 16 <sup>th</sup> Feb – 20 <sup>th</sup> Feb | INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC. |
| 2                                                                                      | 23 <sup>rd</sup> Feb – 27 <sup>th</sup> Feb | CLASSES & OBJECTS                                                              |
| 3                                                                                      | 02 <sup>nd</sup> Mar – 06 <sup>th</sup> Mar | METHODS & MODULAR PROGRAMMING                                                  |
| 4                                                                                      | 09 <sup>th</sup> Mar – 13 <sup>th</sup> Mar | JAVA FUNDAMENTALS <b>ASSIGNMENT #1 &amp; QUIZ #1</b>                           |
| 5                                                                                      | 16 <sup>th</sup> Mar – 20 <sup>th</sup> Mar | INHERITANCE                                                                    |
| 6                                                                                      | 23 <sup>rd</sup> Mar – 27 <sup>th</sup> Mar | POLYMORPHISM                                                                   |
| 7                                                                                      | 30 <sup>th</sup> Mar – 03 <sup>rd</sup> Apr | INTERFACES <b>ASSIGNMENT #2 &amp; QUIZ #2</b>                                  |
| 8                                                                                      | 06 <sup>th</sup> Apr – 10 <sup>th</sup> Apr | EXCEPTION HANDLING                                                             |
| <b>← MID TERM EXAMINATIONS (13<sup>th</sup> APR 26 – 17<sup>th</sup> APR 2026) →</b>   |                                             |                                                                                |
| 9                                                                                      | 20 <sup>th</sup> Apr – 24 <sup>th</sup> Apr | COLLECTIONS FRAMEWORK                                                          |
| 10                                                                                     | 27 <sup>th</sup> Apr – 01 <sup>st</sup> May | FILE HANDLING                                                                  |
| 11                                                                                     | 04 <sup>th</sup> May – 08 <sup>th</sup> May | GUI PROGRAMMING                                                                |
| 12                                                                                     | 11 <sup>th</sup> May – 15 <sup>th</sup> May | APPLICATION DEVELOPMENT <b>ASSIGNMENT #3 &amp; QUIZ #3</b>                     |
| 13                                                                                     | 18 <sup>th</sup> May – 22 <sup>nd</sup> May | DESIGN PRINCIPLES & PATTERNS                                                   |
| 14                                                                                     | 25 <sup>th</sup> May – 29 <sup>th</sup> May | ADVANCED OOP CONCEPTS                                                          |
| 15                                                                                     | 01 <sup>st</sup> Jun – 5 <sup>th</sup> Jun  | CASE STUDIES AND INDUSTRY PRACTICES <b>ASSIGNMENT #4 &amp; QUIZ #4</b>         |
| 16                                                                                     | 08 <sup>th</sup> Jun – 12 <sup>th</sup> Jun | PRESENTATIONS & REVISION WEEK                                                  |
| <b>← FINAL TERM EXAMINATIONS (15<sup>th</sup> JUN 26 – 19<sup>th</sup> JUN 2026) →</b> |                                             |                                                                                |

**Note**

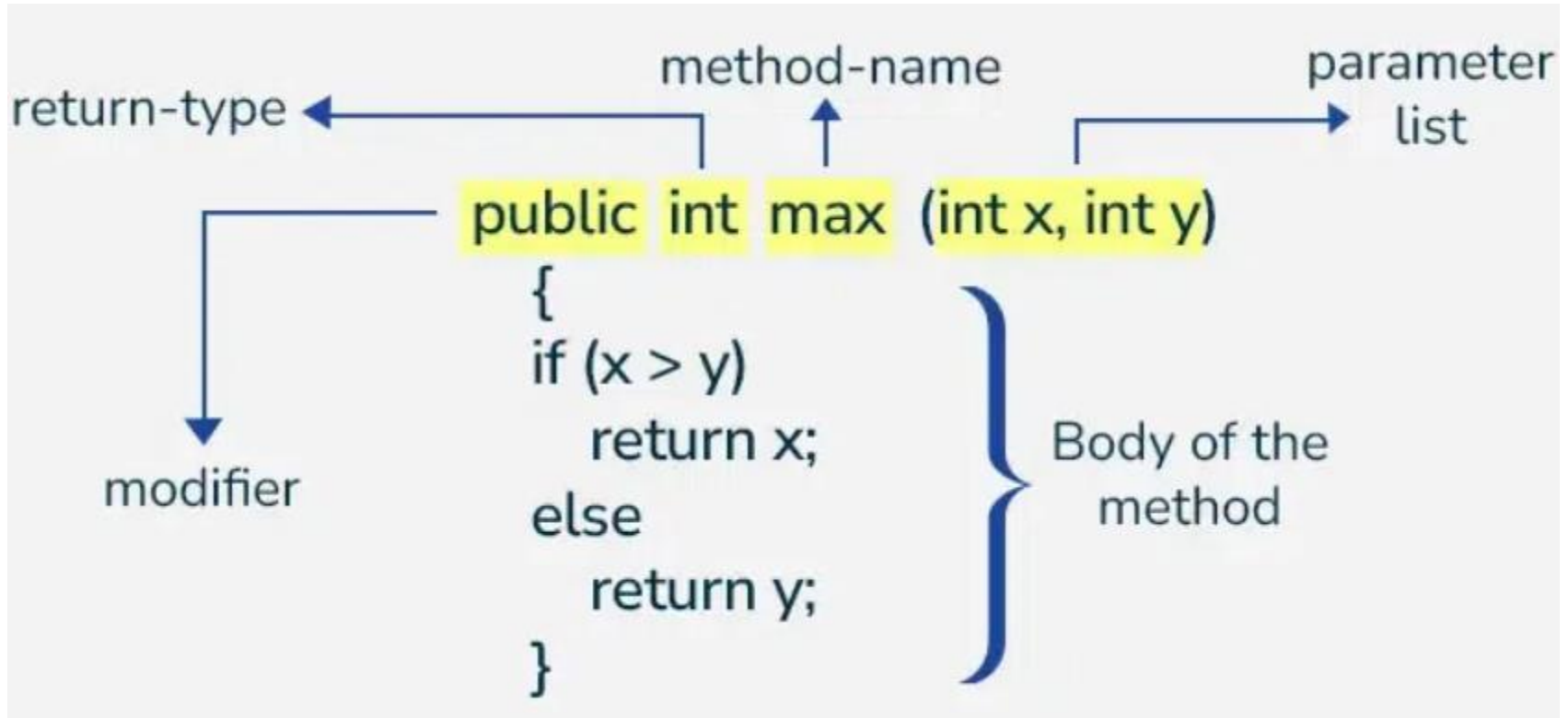
- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

# WEEK 03 - AGENDA

- Methods & Modular Programming

<https://onecompiler.com/java>

# JAVA METHODS



# JAVA METHODS



Code Reusability



Modularity

Why Use Methods



Readability



Maintainability

# JAVA METHODS

- **Definition**

- A method is a block of code that performs a specific task.
- Declared inside a class.

```
int sum(int a, int b) {  
    return a + b;  
}
```

- **Return Type**

- Specifies the type of value the method returns (int, String, void, etc.).
- void means no value is returned.

```
void greet() {  
    System.out.println("Hello");  
}
```

- **Method Signature**

- Combination of method name and parameter list.
- Used by the compiler to differentiate methods (important in overloading).

```
int add(int x, int y) {  
    return x + y;  
} // signature: add(int, int)
```



- **Parameters**

- Input values passed to methods.
- Can be primitive types or objects.
- Support varargs (...) for variable-length arguments.

```
void printName(String name) {  
    System.out.println(name);  
}
```

```
void showNumbers(int... nums) {  
    for(int n : nums)  
        System.out.println(n);  
}
```

# JAVA METHODS

- **Control visibility**
  - public, private, protected, or default (package-private).

```
public void display() {}  
private void helper() {}
```

- **Static vs Instance Methods**

- Static methods belong to the class, can be called without creating an object.
- Instance methods require an object of the class.

```
static void staticMethod() {}  
void instanceMethod() {}
```

# JAVA METHODS

- Static Method

```
public class Harmonic
{
    public static double harmonic(int n)
    {
        double sum = 0.0;
        for (int i = 1; i <= n; i++)
            sum += 1.0/i;
        return sum;
    }

    public static void main(String[] args)
    {
        for (int i = 0; i < args.length; i++)
        {
            int arg = Integer.parseInt(args[i]);
            double value = harmonic(arg);
            StdOut.println(value);
        }
    }
}
```

# JAVA METHODS

- Static Method

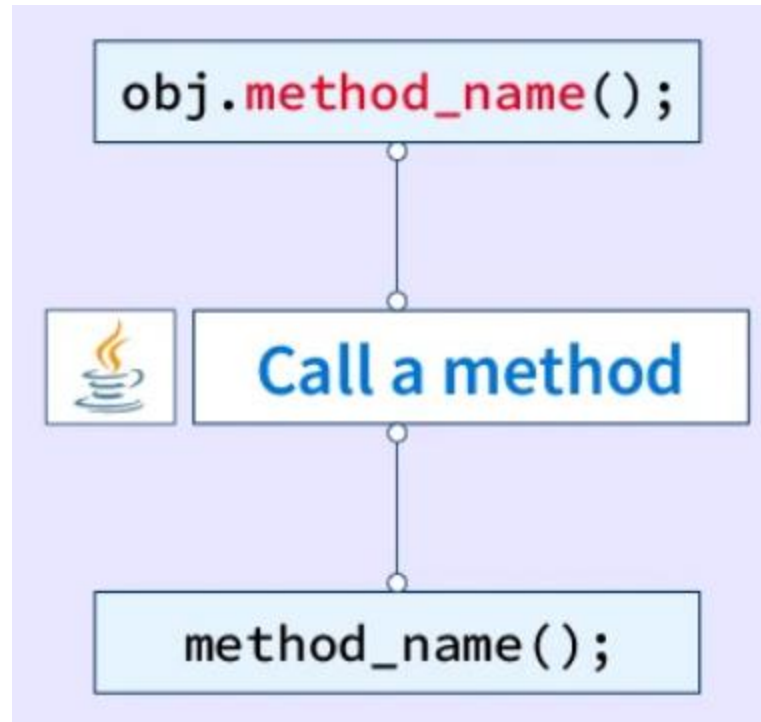
```
// BSE 2A/2B/2C
class Utility {
    // Static method (belongs to the class, not objects)
    static int square(int x) {
        return x * x;
    }
}

public class Demo {
    public static void main(String[] args) {
        // Call static method directly using class name

        int result = Utility.square(5);
        System.out.println(result); // Output: 25
    }
}
```

# JAVA METHODS

- **Instance Method**



# JAVA METHODS

- **Instance Method**

```
class Calculator {    // BSE 2A/2B/2C
    // Instance method (belongs to the object)
    int multiply(int a, int b) {
        return a * b;
    }
}

public class Demo {
    public static void main(String[] args) {
        // Create an object of Calculator
        Calculator calc = new Calculator();

        // Call instance method using the object
        int result = calc.multiply(3, 4);
        System.out.println(result); // Output: 12
    }
}
```

# JAVA METHODS

```
class MathUtils {    // BSE 2A/2B/2C
    // Static method
    static int square(int x) {
        return x * x;
    }

    // Instance method
    int cube(int x) {
        return x * x * x;
    }
}

public class Demo {
    public static void main(String[] args) {
        // Calling static method without object
        System.out.println(MathUtils.square(5)); //works directly

        // Calling instance method requires object
        MathUtils utils = new MathUtils();
        System.out.println(utils.cube(5));        //needs object
    }
}
```



- **Method Overloading**

- Same method name, different parameter lists.
- Provides flexibility for different input types.

```
int multiply(int a, int b) {  
    return a * b;  
}  
double multiply(double a, double b) {  
    return a * b;  
}
```

```
class Calculator {    // BSE 2A/2B/2C
    // Method with two int parameters
    int add(int a, int b) {
        return a + b;
    }

    // Overloaded method with three int parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method with double parameters
    double add(double a, double b) {
        return a + b;
    }
}

public class Demo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(2, 3));           // Output: 5
        System.out.println(calc.add(2, 3, 4));        // Output: 9
        System.out.println(calc.add(2.5, 3.5));        // Output: 6.0
    }
}
```

# KEY ASPECTS

- **Method Overriding**

- Subclass provides a specific implementation of a method defined in its superclass.
- Enables runtime polymorphism.

```
class Animal {  
    void sound() {  
        System.out.println("Generic");  
    }  
}  
  
class Dog extends Animal {  
    void sound()  
    { System.out.println("Bark");  
    }  
}
```

```
class Animal {    // BSE 2A/2B/2C
    // Method in parent class
    void sound() {
        System.out.println("Generic animal sound");
    }
}

class Dog extends Animal {
    // Overriding the parent method
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}

public class Demo {
    public static void main(String[] args) {
        Animal a1 = new Dog();    // Polymorphism
        Animal a2 = new Cat();

        a1.sound();    // Output: Bark
        a2.sound();    // Output: Meow
    }
}
```

- **Constructors**

- Constructors initialize objects when they are created with the *new* keyword, have no return type, not even void, and share the class name.
- The *new* keyword automatically calls the constructor and returns the object itself.
- If you try to specify a return type in a constructor, it becomes a regular method, not a constructor.

```
class Student {  
    Student() {  
        System.out.println("Constructor");  
    }  
    void study() {  
        System.out.println("Method");  
    }  
}
```

# JAVA METHODS

- Constructors

```
class Student {    // BSE 2A/2B/2C
    String name;
    int age;

    // Constructor method
    Student(String n, int a) {
        name = n;
        age = a;
    }

    void display() {
        System.out.println(name + " is " + age + " years old.");
    }
}

// Creating objects using the constructor
Student s1 = new Student("Ali", 20);
Student s2 = new Student("Sara", 22);

s1.display();    // Output: Ali is 20 years old.
s2.display();    // Output: Sara is 22 years old.
```

- **Final Methods**

- Declared with **final** keyword.
- Cannot be overridden by subclasses.

```
final void cannotOverride() {}
```

# JAVA METHODS

- Final Methods

```
class Parent {    // BSE 2A/2B/2C
    // Final method - cannot be overridden
    final void showMessage() {
        System.out.println("This is a final method.");
    }
}

class Child extends Parent {
    // Not allowed: cannot override final method
    // void showMessage() { System.out.println("Trying to override"); }
}

public class Demo {
    public static void main(String[] args) {
        Child c = new Child();
        c.showMessage(); // Output: This is a final method.
    }
}
```



- **Abstract Methods**
  - Declared in abstract classes or interfaces.
  - No body; must be implemented by subclasses.

```
abstract class Shape {  
    abstract void draw();  
}
```

```
// Abstract class
abstract class Shape {    // BSE 2A/2B/2C
    // Abstract method (no body)
    abstract void draw();
}

// Subclass must implement the abstract method
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Square extends Shape {
    void draw() {
        System.out.println("Drawing a Square");
    }
}

public class Demo {
    public static void main(String[] args) {
        Shape s1 = new Circle(); // Polymorphism
        Shape s2 = new Square();

        s1.draw(); // Output: Drawing a Circle
        s2.draw(); // Output: Drawing a Square
    }
}
```

# JAVA METHODS

```
1 // BSE 2A/2B/2C
2 // Parent class
3 class Parent {
4     void greet() {
5         System.out.println("Hello from Parent!");
6     }
7 }
8
9 // Child class
10 class Child extends Parent {
11     @Override
12     void greet() {
13         System.out.println("Hello from Child!");
14         super.greet(); // <-- calls the Parent's greet method
15     }
16 }
17
18 // Main class to test
19 public class Main {
20     public static void main(String[] args) {
21         Child child = new Child();
22         child.greet(); // Calls Child's greet, which also calls Parent's greet
23     }
24 }
```

# JAVA METHODS

```
1 // BSE 2A/2B/2C
2 // Parent class
3 class Parent {
4     void greet() {
5         System.out.println("Hello from Parent!");
6     }
7 }
8
9 // Child class
10 class Child extends Parent {
11     // Same method name as in Parent
12     void greet() {
13         System.out.println("Hello from Child!");
14     }
15 }
16
17 // Main class
18 public class Main {
19     public static void main(String[] args) {
20         Child child = new Child(); // create child object
21         child.greet();             // call greet()
22     }
23 }
24
```

- **Recursion**
  - A method calling itself to solve problems (e.g., factorial, Fibonacci).
  - Requires a base case to avoid infinite loops.

```
int factorial(int n) {  
    return (n==0) ? 1 : n * factorial(n-1);  
}
```

- **Synchronized Methods**

- Used in multithreading to prevent concurrent access issues.
- Ensures only one thread executes the method at a time.

```
synchronized void safeIncrement() { /* thread-safe code */ }
```

- **Native Methods**

- Declared with native keyword.
- Implemented in another language (like C/C++), often for performance or system-level tasks.

```
native void systemCall(); // implemented in C/C++
```

*To be continued...*



```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```